ReadyOS SHIM Architecture: 0.2.4 Runtime Reality

Implementation-backed report for the active launcher + shim + REU model. This canonical report is intentionally fuller than the short summary docs: it keeps the resident-byte map, the annotated shim routine walkthrough, the profile-driven catalog/release context, and the ABI-vs-implementation nuances that are easy to lose in condensed summaries. This report preserves the previous local snapshot values and adds current `0.2.4` map-backed values after the reserved launcher-overlay REU bank change. Runtime debug suffix letters are intentionally not used as documentation versions.

Executive Snapshot

RELEASE	SHIM WINDOW	SHIM SIZE	APP WINDOW	SNAPSHOT SIZE
`0.2.4`	`$C800–$C9FF`	`512 bytes`	`$1000–$C5FF`	`$B600`

What changed from 0.2.3 to 0.2.4:

ReadyOS has a physical REU start bank called `Start` (`READYOS_REU_BANK_SKIP`). The fixed ReadyOS region now uses the first three banks relative to that start point: `Start+0` for system/global state, `Start+1` for the launcher snapshot/resume bank, and `Start+2` for the future launcher overlay bank.

Only after those three ReadyOS/launcher banks do the catalog app slots begin. App-visible logical app slots are still numbered `1..23`, but their physical backing starts at `Start+3`: app slot `1` is `Start+3`, app slot `2` is `Start+4`, and app slot `23` is `Start+25`. Dynamic allocation begins after that at `Start+26`.

The shim now performs that mapping in `$C960–$C99F` via `reu_setup_logical`. The old `$C818`/`log_byte` diagnostic path remains only as a legacy entry until its ring storage moves away from active code bytes.

Canonical C64 RAM Map

`$1000–$C5FF`	Active app snapshot/load window	`$B600`
`$C600–$C7FF`	REU metadata RAM, clipboard tables, debug bytes, hotkey bindings	`512 B`
`$C800–$C9FF`	Resident shim jump table, globals, and swap/load routines	`512 B`
`$D000–$DFFF`	I/O and hardware registers	`4 KB`

Shared RAM highlights

Address	Purpose
`$C600–$C6FF`	REU allocation table
`$C700`	REU manager magic
`$C702 / $C710`	clipboard count / table metadata
`$C7A0–$C7DF / $C7F0`	ReadyShell RAM debug ring + head
`$C7E0–$C7E8`	global hotkey bindings

Why the layout matters

- Everything in `$1000–$C5FF` is replaceable.
- The shim survives all loads because it lives above the snapshot window.
- The hardware stack survives preload and enables the shim's **RTS**-based return path.
- Metadata in `$C600–$C9FF` stays visible without fetching a management bank from REU.

Resident Shim Byte Map

Page 1: `$C800–$C8FF`

`$C800–$C81F`
jump/helper entries including legacy `$C818` log-byte helper jump

`$C820–$C83F`
shim data ABI: target bank, filename, load end, current bank, last saved, bitmap bytes, storage drive, log index

`$C840–$C85F`
`load_disk`

`$C860–$C87F`
`load_reu`

`$C880–$C8DF`
`preload` sandwich

`$C8E0–$C8EF`
`stash_to_bank`

`$C8F0–$C8FF`
`fetch_bank`

Page 2: `$C900–$C9FF`

`$C900–$C93F`
`return_to_launcher`

`$C940–$C95F`
`switch_app`

`$C960–$C99F`
`reu_setup_logical` maps logical to physical REU banks

`$C9A0–$C9BF`
`reu_setup` direct physical-bank helper

`$C9C0–$C9DF`
`set_bitmap`

`$C9E0–$C9FF`
`log_byte` legacy helper; unsafe until ring moves

Jump / helper entry block

Address	Entry	Meaning
`$C800`	load disk + run	KERNAL load app and jump to `$1000`
`$C803`	load REU + run	fetch snapshot image and jump to `$1000`
`$C806`	run app	direct jump to `$1000`
`$C809`	preload	load app to REU and return to launcher caller
`$C80C`	return to launcher	stash current app and restore the launcher bank at `Start+1`
`$C80F`	switch app	stash current app and fetch target app
`$C812`	reserved helper slot	currently a placeholder jump slot in the resident shim, not a live stash helper
`$C815`	fetch helper	helper entry into REU fetch path
`$C818`	legacy log-byte helper	retained ABI-shaped entry; unsafe until the ring moves away from active `$C980–$C99F` code bytes

Shim data ABI

Address	Field	Active use
`$C820`	`target_bank`	preload / switch / fetch target logical bank
`$C821`	`filename_len`	launcher-written filename metadata
`$C822–$C823`	`app_size` scratch	launcher scratch / optional convenience field; resident shim does not read it
`$C824–$C82F`	`filename[12]`	12-byte app token buffer for disk load paths
`$C830–$C831`	`load_end`	actual KERNAL `LOAD` end address captured by preload
`$C832–$C833`	unused	reserved padding
`$C834`	`current_bank`	currently running logical bank; read directly by apps and TUI helpers
`$C835`	`last_saved`	recovery hint for interrupted return path
`$C836–$C838`	bitmap lo/hi/xhi	loaded-state bits for the launcher bank plus app slots `1..23`
`$C839`	`storage_drive`	shared file-dialog default drive
`$C83A`	`log_index`	legacy ring head; `$C980–$C99F` is no longer safe ring storage
`$C83B`	`reu_bank_skip`	compiled physical REU bank offset for the ReadyOS region
`$C83C–$C83F`	reserved	unused padding
`$C84D`	device immediate	`load_disk` launcher-patched drive byte
`$C89C`	device immediate	`preload` launcher-patched drive byte

Current preload result: launcher scratch may still use `$C822–$C823`, but the real preload result is the KERNAL `LOAD` end address saved at `$C830–$C831`.

Annotated Resident Shim Walkthrough

This is a complete report-local annotation of the resident shim byte image. The comments here use the clearer 0.2.4 Start-relative terminology; the emitted byte sequence is checked against `src/boot/readyos_shim.inc` and still covers the jump table, every data ABI byte, all live routines, and all padding bytes in the 512-byte `$C800–$C9FF` image.

```
;-----------------------------------------------------------------------------
; Shared ReadyOS shim image ($C800–$C9FF, 512 bytes)
; This file is the canonical shim byte layout used by both the disk boot path
; and the EasyFlash cartridge flavor. Keep it identical across all variants.
;-----------------------------------------------------------------------------

;-----------------------------------------------------------------------------
; Page 1: $C800–$C8FF – Jump table, data, and helper routines
;-----------------------------------------------------------------------------

; $C800–$C817: Jump Table (24 bytes, 8 entries)
jt_load_disk    = $C800     ; Load from disk, run
jt_load_reu     = $C803     ; Fetch from REU, run
jt_run_app      = $C806     ; Just run app
jt_preload      = $C809     ; Preload to REU, return
jt_return       = $C80C     ; Return to launcher
jt_switch       = $C80F     ; Switch to another app
jt_stash_cur    = $C812     ; Helper: stash current app
jt_fetch_bank   = $C815     ; Helper: fetch from bank in A

.byte $4C, $40, $C8         ; $C800: JMP load_disk ($C840)
.byte $4C, $60, $C8         ; $C803: JMP load_reu ($C860)
.byte $4C, $00, $10         ; $C806: JMP $1000 (run app)
.byte $4C, $80, $C8         ; $C809: JMP preload ($C880)
.byte $4C, $00, $C9         ; $C80C: JMP return_to_launcher ($C900)
.byte $4C, $40, $C9         ; $C80F: JMP switch_app ($C940)
.byte $4C, $C0, $C8         ; $C812: JMP stash_current ($C8C0) – placeholder
.byte $4C, $F0, $C8         ; $C815: JMP fetch_bank ($C8F0)

; $C818: JMP log_byte ($C9E0)
; Legacy diagnostic entry. The old $C980–$C99F ring now overlaps active
; reu_setup_logical bytes, so callers must not use this unless the ring moves.
.byte $4C, $E0, $C9         ; $C818: JMP log_byte
.byte $00,$00,$00,$00,$00   ; $C81B–$C81F: Padding

; $C820–$C83F: Data Area (32 bytes)
; $C820: target_bank – logical bank to load/switch to
; $C821: filename_len
; $C822–$C823: unused
; $C824–$C82F: filename (12 bytes)
; $C830: load_end_lo (KERNAL LOAD end addr low byte, saved by preload)
; $C831: load_end_hi (KERNAL LOAD end addr high byte, saved by preload)
; $C832–$C833: unused
; $C834: current_bank – currently running logical bank
; $C835: last_saved
; $C836: reu_bitmap_lo (logical banks 0–7)
; $C837: reu_bitmap_hi (logical banks 8–15)
; $C838: reu_bitmap_xhi (logical banks 16–23)
; $C839: storage_drive – shim-global default storage drive for D8/D9 app dialogs
;        This persists across app switches and is shared by apps that use the
;        common file-dialog default-drive contract.
; $C83A: log_index – legacy debug byte ring head
; $C83B: reu_bank_skip – physical 64K banks reserved before ReadyOS bank start
; $C83C–$C83F: reserved

.byte $00                   ; $C820: target_bank
.byte $08                   ; $C821: filename_len
.byte $00, $00              ; $C822–$C823: unused
.byte "LAUNCHER    "        ; $C824–$C82F: filename (12 bytes)
.byte $00,$00               ; $C830–$C831: load_end_lo, load_end_hi
.byte $00,$00               ; $C832–$C833: unused
.byte $00                   ; $C834: current_bank
.byte $FF                   ; $C835: last_saved
.byte $00                   ; $C836: reu_bitmap_lo
.byte $00                   ; $C837: reu_bitmap_hi
.byte $00                   ; $C838: reu_bitmap_xhi
.byte $08                   ; $C839: storage_drive (default drive 8)
.byte $00                   ; $C83A: log_index (legacy debug buffer)
.byte READYOS_REU_BANK_SKIP ; $C83B: reu_bank_skip (compiled into boot image)
.byte $00,$00,$00,$00       ; $C83C–$C83F: reserved

;-----------------------------------------------------------------------------
; $C840: load_disk – Load app from disk and run (32 bytes)
;-----------------------------------------------------------------------------
.byte $AD, $21, $C8         ; LDA $C821 (filename len)
.byte $A2, $24              ; LDX #$24
.byte $A0, $C8              ; LDY #$C8 (filename at $C824)
.byte $20, $BD, $FF         ; JSR $FFBD (SETNAM)
.byte $A9, $00              ; LDA #0
.byte $A2, $08              ; LDX #8
.byte $A0, $01              ; LDY #1
.byte $20, $BA, $FF         ; JSR $FFBA (SETLFS)
.byte $A9, $00              ; LDA #0
.byte $20, $D5, $FF         ; JSR $FFD5 (LOAD)
.byte $4C, $00, $10         ; JMP $1000
.byte $00,$00,$00,$00,$00   ; Padding to $C860

;-----------------------------------------------------------------------------
; $C860: load_reu – Fetch app from REU and run (32 bytes)
;-----------------------------------------------------------------------------
.byte $AD, $20, $C8         ; LDA $C820 (target bank)
.byte $20, $F0, $C8         ; JSR fetch_bank ($C8F0)
.byte $4C, $00, $10         ; JMP $1000
.byte $00,$00,$00,$00,$00,$00,$00,$00
.byte $00,$00,$00,$00,$00,$00,$00,$00
.byte $00,$00,$00,$00,$00,$00,$00

;-----------------------------------------------------------------------------
; $C880: preload – Load to REU, return to launcher
; Called via JSR $C809
; DEBUG: writes markers to $C007–$C00C
;-----------------------------------------------------------------------------
.byte $A9, $AA              ; LDA #$AA (170 = "preload entered")
.byte $8D, $07, $C0         ; STA $C007

.byte $A9, $00              ; LDA #0 (select launcher bank at Start+1)
.byte $20, $E0, $C8         ; JSR stash_to_bank ($C8E0)

.byte $A9, $BB              ; LDA #$BB (187 = "launcher stashed")
.byte $8D, $08, $C0         ; STA $C008

.byte $AD, $21, $C8         ; LDA $C821 (len)
.byte $A2, $24              ; LDX #$24
.byte $A0, $C8              ; LDY #$C8 (filename at $C824)
.byte $20, $BD, $FF         ; JSR SETNAM

.byte $A9, $00              ; LDA #0
.byte $A2, $08              ; LDX #8
.byte $A0, $01              ; LDY #1
.byte $20, $BA, $FF         ; JSR SETLFS

.byte $A9, $CC              ; LDA #$CC (204 = "about to LOAD")
.byte $8D, $09, $C0         ; STA $C009

.byte $A9, $00              ; LDA #0
.byte $20, $D5, $FF         ; JSR LOAD

.byte $8E, $30, $C8         ; STX $C830 (end addr lo)
.byte $8C, $31, $C8         ; STY $C831 (end addr hi)

.byte $AD, $20, $C8         ; LDA $C820 (target bank)
.byte $20, $E0, $C8         ; JSR stash_to_bank ($C8E0)

.byte $A9, $EE              ; LDA #$EE (238 = "app stashed")
.byte $8D, $0B, $C0         ; STA $C00B

.byte $AD, $20, $C8         ; LDA $C820 (target bank)
.byte $20, $C0, $C9         ; JSR set_bitmap ($C9C0)

.byte $A9, $00              ; LDA #0 (select launcher bank at Start+1)
.byte $20, $F0, $C8         ; JSR fetch_bank ($C8F0)

.byte $A9, $FF              ; LDA #$FF (255 = "launcher restored")
.byte $8D, $0C, $C0         ; STA $C00C

.byte $60                   ; RTS

.byte $00,$00,$00,$00,$00,$00,$00,$00,$00,$00
.byte $00,$00,$00,$00,$00,$00,$00,$00

;-----------------------------------------------------------------------------
; $C8E0: stash_to_bank – Stash $1000–$C5FF to logical REU bank in A (16 bytes)
;-----------------------------------------------------------------------------
.byte $20, $60, $C9         ; JSR reu_setup_logical ($C960)
.byte $A9, $90              ; LDA #$90 (STASH command)
.byte $8D, $01, $DF         ; STA $DF01 – execute transfer
.byte $60                   ; RTS
.byte $00,$00,$00,$00,$00,$00,$00,$00

;-----------------------------------------------------------------------------
; $C8F0: fetch_bank – Fetch logical REU bank in A to $1000–$C5FF (16 bytes)
;-----------------------------------------------------------------------------
.byte $20, $60, $C9         ; JSR reu_setup_logical ($C960)
.byte $A9, $91              ; LDA #$91 (FETCH command)
.byte $8D, $01, $DF         ; STA $DF01 – execute transfer
.byte $60                   ; RTS
.byte $00,$00,$00,$00,$00,$00,$00,$00

;-----------------------------------------------------------------------------
; Page 2: $C900–$C9FF – Main routines
;-----------------------------------------------------------------------------

;-----------------------------------------------------------------------------
; $C900: return_to_launcher (64 bytes)
;-----------------------------------------------------------------------------
.byte $AD, $34, $C8         ; LDA $C834 (current logical bank)
.byte $20, $E0, $C8         ; JSR stash_to_bank ($C8E0)
.byte $AD, $34, $C8         ; LDA $C834 (current logical bank)
.byte $20, $C0, $C9         ; JSR set_bitmap ($C9C0)
.byte $AD, $34, $C8         ; LDA $C834
.byte $8D, $35, $C8         ; STA $C835
.byte $A9, $00              ; LDA #0 (select launcher bank at Start+1)
.byte $20, $F0, $C8         ; JSR fetch_bank ($C8F0)
.byte $A9, $00              ; LDA #0 (select launcher bank at Start+1)
.byte $8D, $34, $C8         ; STA $C834
.byte $4C, $00, $10         ; JMP $1000
.byte $00,$00,$00,$00,$00,$00,$00,$00
.byte $00,$00,$00,$00,$00,$00,$00,$00
.byte $00,$00,$00,$00,$00,$00,$00,$00
.byte $00,$00,$00,$00,$00,$00,$00,$00
.byte $00

;-----------------------------------------------------------------------------
; $C940: switch_app (64 bytes)
;-----------------------------------------------------------------------------
.byte $AD, $34, $C8         ; LDA $C834 (current logical bank)
.byte $20, $E0, $C8         ; JSR stash_to_bank ($C8E0)
.byte $AD, $34, $C8         ; LDA $C834 (current logical bank)
.byte $20, $C0, $C9         ; JSR set_bitmap ($C9C0)
.byte $AD, $20, $C8         ; LDA $C820 (target logical bank)
.byte $20, $F0, $C8         ; JSR fetch_bank ($C8F0)
.byte $AD, $20, $C8         ; LDA $C820
.byte $8D, $34, $C8         ; STA $C834
.byte $4C, $00, $10         ; JMP $1000
.byte $00,$00,$00,$00,$00

;-----------------------------------------------------------------------------
; $C960: reu_setup_logical – Map logical bank to physical, then setup REU regs
;
; Physical ReadyOS region layout:
;   Start+0: system/global bank
;   Start+1: launcher snapshot/resume bank
;   Start+2: reserved launcher overlay bank
;   Start+3..Start+25: physical backing banks for app slots 1..23
;-----------------------------------------------------------------------------
.byte $C9, $00              ; CMP #0 (select launcher bank at Start+1?)
.byte $D0, $09              ; BNE app_bank
.byte $AD, $3B, $C8         ; LDA $C83B (reu_bank_skip)
.byte $18                   ; CLC
.byte $69, $01              ; ADC #$01 (launcher is Start+1)
.byte $4C, $74, $C9         ; JMP store_physical_bank
.byte $18                   ; app_bank: CLC
.byte $6D, $3B, $C8         ; ADC $C83B (logical app bank + skip)
.byte $18                   ; CLC
.byte $69, $02              ; ADC #$02 (global + launcher overlay reserve)
.byte $8D, $06, $DF         ; STA $DF06 (physical bank)
.byte $A9, $00              ; LDA #$00
.byte $8D, $02, $DF         ; STA $DF02 (C64 addr lo = $00)
.byte $A9, $10              ; LDA #$10
.byte $8D, $03, $DF         ; STA $DF03 (C64 addr hi = $10, so $1000)
.byte $A9, $00              ; LDA #$00
.byte $8D, $04, $DF         ; STA $DF04 (REU addr lo)
.byte $8D, $05, $DF         ; STA $DF05 (REU addr hi)
.byte $8D, $07, $DF         ; STA $DF07 (len lo = $00)
.byte $A9, $B6              ; LDA #$B6
.byte $8D, $08, $DF         ; STA $DF08 (len hi = $B6, so $B600 bytes)
.byte $60                   ; RTS

.byte $00,$00,$00,$00,$00,$00,$00,$00,$00,$00,$00,$00,$00,$00,$00

;-----------------------------------------------------------------------------
; $C9A0: reu_setup – Set up REU registers for a physical bank in A
;-----------------------------------------------------------------------------
.byte $8D, $06, $DF         ; STA $DF06 (bank)
.byte $A9, $00              ; LDA #$00
.byte $8D, $02, $DF         ; STA $DF02 (C64 addr lo = $00)
.byte $A9, $10              ; LDA #$10
.byte $8D, $03, $DF         ; STA $DF03 (C64 addr hi = $10, so $1000)
.byte $A9, $00              ; LDA #$00
.byte $8D, $04, $DF         ; STA $DF04 (REU addr lo)
.byte $8D, $05, $DF         ; STA $DF05 (REU addr hi)
.byte $8D, $07, $DF         ; STA $DF07 (len lo = $00)
.byte $A9, $B6              ; LDA #$B6
.byte $8D, $08, $DF         ; STA $DF08 (len hi = $B6, so $B600 bytes)
.byte $60                   ; RTS
.byte $00,$00

;-----------------------------------------------------------------------------
; $C9C0: set_bitmap – Set bit for bank in A in reu_bitmap ($C836–$C838)
;-----------------------------------------------------------------------------
.byte $C9, $18              ; CMP #$18 (only logical banks 0–23 are tracked)
.byte $B0, $17              ; BCS done
.byte $AA                   ; TAX – preserve full bank
.byte $29, $07              ; AND #$07 – bit index within byte
.byte $A8                   ; TAY – Y = bit index
.byte $8A                   ; TXA – restore full bank
.byte $4A                   ; LSR A
.byte $4A                   ; LSR A
.byte $4A                   ; LSR A
.byte $AA                   ; TAX – X = byte offset 0..2
.byte $A9, $01              ; LDA #$01
.byte $88                   ; DEY
.byte $30, $03              ; BMI store_bit
.byte $0A                   ; ASL A
.byte $10, $FA              ; BPL shift_loop
.byte $1D, $36, $C8         ; ORA $C836,X
.byte $9D, $36, $C8         ; STA $C836,X
.byte $60                   ; RTS
.byte $00,$00,$00,$00

;-----------------------------------------------------------------------------
; $C9E0: log_byte – Legacy debug marker writer
; WARNING: $C980–$C99F now overlaps active reu_setup_logical code bytes.
; This entry is kept for ABI shape only; do not call it unless the ring moves.
;-----------------------------------------------------------------------------
.byte $48                   ; PHA – save the byte to log
.byte $8E, $FC, $00         ; STX $00FC – save X to ZP temp
.byte $AE, $3A, $C8         ; LDX $C83A – get log_index
.byte $68                   ; PLA – get byte back
.byte $9D, $80, $C9         ; STA $C980,X – write to buffer
.byte $E8                   ; INX – increment index
.byte $E0, $20              ; CPX #$20 – wrap at 32
.byte $D0, $02              ; BNE +2 (skip reset)
.byte $A2, $00              ; LDX #$00 – reset to 0
.byte $8E, $3A, $C8         ; STX $C83A – store new index
.byte $AE, $FC, $00         ; LDX $00FC – restore X
.byte $60                   ; RTS
.byte $00,$00,$00,$00,$00,$00,$00
```

REU Ownership and Clipboard Accuracy

Current REU layout relative to Start

Bank(s)	Role
`Start+0`	ReadyOS system/global bank
`Start+1`	launcher snapshot/resume bank
`Start+2`	launcher overlay reserve bank
`Start+3..Start+25`	catalog app slots: logical app slot 1 begins at Start+3, through logical app slot 23 at Start+25
`Start+26+`	dynamic allocator pool; after ReadyOS, launcher, and fixed app slots
`$40`	ReadyShell overlay cache bank for overlays 1, 2, 3, and 5
`$41`	ReadyShell overlay cache bank for overlays 4, 6, 7, and 8
`$43`	ReadyShell REU debug ring
`$48`	ReadyShell scratch, registry, metadata, and value arena

Logical bank 0 split

Offset	Meaning
`$0000–$B5FF`	launcher snapshot image of C64 RAM `$1000–$C5FF`
`$B600–$FFFF`	launcher resume tail (`LauncherResumeV1` + header)

Clipboard storage reality

- **Clipboard metadata in RAM:** `$C702` count, `$C710` table.
- **Per-item table layout:** each item stores REU bank id, type, and size.
- **Allocator behavior:** `clip_copy()` calls `reu_alloc_bank(REU_CLIPBOARD)`, so payload banks come from the dynamic pool starting at `Start+26`.
- **Legacy symbolic contract:** some older headers/spec fields still name bank `1` as a clipboard bank, but active payload storage is not single-bank packed there.

Truthful current wording: clipboard metadata is shared in C64 RAM, and clipboard payloads still use one dynamic REU bank per item. The single-bank packed clipboard design has not been implemented yet.

```
REU_APP_SNAPSHOT_SIZE = $B600
REU_RESUME_OFF        = $B600
REU_RESUME_TAIL_SIZE  = $4A00
tail end              = $10000
```

`apps.cfg` and Launcher Lifecycle

Disk-based SKUs

- Human-edited source: selected `cfg/profiles/<profile>.ini` catalog
- Generator/orchestrator: `build_support/readyos_profiles.py`
- Disk payload: `apps.cfg` on drive `8`
- Boot loader: `src/boot/boot_asm.s`, linked as the normal disk boot path
- Launcher app: `src/apps/launcher/launcher.c`, with `READYOS_LAUNCHER_VARIANT_EASYFLASH=0`
- Disk-only launcher code includes the file dialog and storage-device helpers, guarded by `#if !READYOS_LAUNCHER_VARIANT_EASYFLASH`
- Catalog read mode: `apps.cfg,s,r`
- Parsed fields: drive, PRG token, display label, optional hotkey slot, description
- Assigned at runtime by the launcher: app slot identity and REU preload state

Disk lifecycle

Cold / invalid-resume path
launcher starts → tries launcher-bank resume payload → failure → parse `apps.cfg` from disk → validate → save fresh `LauncherResumeV1`

Normal return path
app returns via shim → launcher image restored from the launcher snapshot bank at `Start+1` → launcher restarts → restores cached state from launcher bank tail → `apps.cfg` usually not reread

Cartridge / EasyFlash SKU

- Human-edited source: `cfg/flavors/readyos_easyflash.ini`
- Catalog generator: `build_support/readyos_easyflash.py catalog-header`
- Generated launcher catalog: `src/generated/launcher_easyflash_catalog.h`
- Generated boot/shim REU config: `src/generated/readyos_easyflash_reu_config.inc`
- Generated cartridge layout: `src/generated/easyflash_layout.inc` and `obj/easyflash_layout.json`
- Custom booter: `src/boot/boot_easyflash_asm.s` plus `src/boot/easyflash_shim.s`
- Launcher app: `src/apps/launcher/launcher_easyflash.c`, which defines `READYOS_LAUNCHER_VARIANT_EASYFLASH=1` and includes `launcher.c`
- ReadyShell cartridge build: `bin/readyshell_easyflash.prg` with `RS_C64_OVERLAY_PRELOADED=1`

Cartridge lifecycle

Before launcher menu
the EasyFlash booter copies the shim, restores the EasyFlash launcher payload into the launcher snapshot bank, and preloads the cartridge catalog apps plus the ReadyShell overlay cache into REU.

Launcher catalog path
the EasyFlash launcher does not open `apps.cfg`. It consumes generated arrays such as `READYOS_EASYFLASH_APP_COUNT`, `readyos_easyflash_prgs`, `readyos_easyflash_labels`, `readyos_easyflash_descs`, `readyos_easyflash_default_slots`, and `readyos_easyflash_app_banks`.

Disk boot never parses `apps.cfg`; the disk launcher does. Cartridge boot never depends on `apps.cfg` at runtime; its app catalog, physical preload plan, and ReadyShell overlay preload plan are generated into the EasyFlash build.

Current release matrix: there are seven active profiles. `precog-dual-d71` is the default full-content verification target and the only one that uses `PREBOOT -> SETD71 -> BOOT`. The other six current profiles use direct `PREBOOT -> BOOT`.

Measured BSS and Headroom Snapshot

The first table preserves the previous local snapshot that originally backed this report. The second table is the current `0.2.4` audit regenerated from the linker `Segment list` in `obj/*.map`. These rows report exact linked runtime ends, exact raw headroom to `$C5FF`, and the even-aligned writable slack left after BSS.

Tightest normal app images (previous local snapshot)

App	Runtime end	BSS bytes	Headroom			
`launcher`	`$C33A`	`1231`	`49`	`5141`	`708`	`709`
`dizzy`	`$BED5`	`1225`	`132`	`6327`	`1834`	`1834`
`tasklist`	`$AE64`	`821`	`151`	`8927`	`6042`	`6043`
`calcplus`	`$AC00`	`1173`	`158`	`1321`	`6654`	`6655`
`cal26`	`$A3EA`	`1869`	`324`	`4803`	`8724`	`8725`
`simplecells`	`$A2A8`	`1722`	`238`	`2142`	`9046`	`9047`
`quicknotes`	`$A0EB`	`944`	`222`	`6031`	`9492`	`9492`
`editor`	`$9859`	`830`	`134`	`9880`	`11686`	`11686`
`simplefiles`	`$94A2`	`1229`	`62`	`4397`	`12636`	`12637`
`clipmgr`	`$9142`	`842`	`46`	`1062`	`13500`	`13501`
`deminer`	`$6F68`	`747`	`46`	`4276`	`22166`	`22167`
`readme`	`$6D23`	`13216`	`46`	`48`	`22748`	`22748`
`game2048`	`$565B`	`304`	`46`	`1203`	`28580`	`28580`
`sysinfo`	`$50B9`	`1064`	`47`	`277`	`30022`	`30022`
`reuviewer`	`$49CD`	`346`	`46`	`54`	`31794`	`31794`
`hexview`	`$47AC`	`498`	`46`	`101`	`32338`	`32339`

Measurement method: runtime end is the highest end address among `STARTUP`, `CODE`, `RODATA`, `DATA`, `INIT`, `ONCE`, and `BSS` in each linker map. **Aligned writable slack** is the even-aligned free span after that runtime end and below the ReadyOS snapshot ceiling. **Raw headroom** is the exact byte gap from the linked runtime end to `$C5FF`. `readyshell` is intentionally omitted from this normal-app table because it uses the overlay linker contract and has a separate resident heap ceiling at `$8DFD`.

Current memory contract

- **Real app snapshot window:** normal ReadyOS apps link and resume inside `$1000–$C5FF`, not `$1000–$BFFF`.
- **Why this is trustworthy:** `cfg/ready_app.cfg` defines `MAIN` as `$1000 + $B600`, and the shim copies exactly `$B600` bytes during REU stash/fetch.
- **Suspend/resume split:** REU bank offsets `$0000–$B5FF` hold the raw app snapshot, and offsets `$B600–$FFFF` hold the per-app resume payload.
- **Report consequence:** app headroom must be measured to `$C5FF`, while logical resume state belongs to the REU tail outside that snapshot.
- **Code/docs alignment:** the boot-loader memory-map comment, linker configs, shim implementation, and verifiers all use the same `$1000–$C5FF` app snapshot window.

Interpreting slack

- **Normal apps:** `cfg/ready_app.cfg` puts BSS directly inside the `$1000–$C5FF` app window.
- **Aligned writable slack is not always heap:** the table reports what is left after BSS and even alignment, but that should not be read as a proven malloc heap unless the app actually initializes and uses it that way.
- **Simple Files / Simple Cells:** those apps use dedicated linker configs with an explicit `__STACKSIZE__ = $0070` reserve below `$C600`, so their slack figures include a deliberate stack guardrail and are not unconstrained heap.
- **Overlay app:** `readyshell` uses `cfg/ready_app_overlay.cfg`, not the normal app linker contract.
- **Historical resident ReadyShell image:** the previous local snapshot ended at `$9B74`, with a much smaller resident image. Those values are retained only for historical comparison.
- **Current resident ReadyShell image:** the current `0.2.4` map has `RODATA 1050`, `DATA 71`, resident BSS `$8780–$8976` (`503` bytes), resident heap `$8978–$8DFD` (`1158` bytes), and overlay load-address bytes at `$8DFE–$8DFF`.
- **True current ReadyShell limit:** overlay payload fit is tighter than the resident heap gap. Current overlay1 is `$8E00–$C0CC` and leaves `1331` bytes; overlay2 leaves `303` bytes; overlay3 leaves `3064` bytes; overlay4 leaves `2452` bytes; overlay5 leaves `4131` bytes; overlay6 leaves `3` bytes; overlay7 leaves `6258` bytes; overlay8 leaves `7737` bytes.
- **Overlay guardrail:** release overlay size is `$3800` and debug overlay size is `$3B00`. The old explicit-C-stack-reserve wording is retained as historical shorthand, but current safety depends on ONCE/BSS ending below `__OVERLAY_LOADADDR__`, overlays loading at `$8E00`, and ReadyShell stack relocation being handled separately.

Transition Flows

Preload sandwich

1. Launcher writes target bank, filename, and drive into shim bytes.
2. Launcher calls `JSR $C809`.
3. Shim stashes launcher image to the launcher bank at `Start+1`.
4. Shim KERNAL-loads target app into `$1000`.
5. Shim saves real load end to `$C830–$C831`.
6. Shim stashes loaded app to target logical bank, sets bitmap, restores launcher, and **RTS**es.

Return to launcher

1. App jumps to `$C80C` through TUI helper.
2. Shim stashes current app to its bank.
3. Shim updates bitmap and records `last_saved`.
4. Shim fetches the launcher bank at `Start+1`.
5. Shim sets `current_bank = 0` and jumps to `$1000`.
6. Launcher restarts and usually restores cached menu/catalog state from the launcher bank tail.

Launch from REU

1. Launcher saves `LauncherResumeV1`.
2. Launcher stashes itself to the launcher bank at `Start+1`.
3. Launcher writes target scratch bytes and sets `current_bank`.
4. Launcher jumps to `$C803`.

Launch from disk

1. Launcher saves `LauncherResumeV1`.
2. Launcher stashes itself to the launcher bank at `Start+1`.
3. Launcher writes filename and patched drive bytes.
4. Launcher sets `current_bank`.
5. Launcher jumps to `$C800`.

Warm Resume and App Nuances

Topic	Reality
Resume header	16-byte `RSM1` header with app id, schema, seq, and payload length
Launcher resume	`LauncherResumeV1` stores parsed catalog plus menu position in the launcher bank tail at physical `Start+1`
Snapshot vs resume	snapshot restores raw runnable image; resume tail restores validated high-level state
Tasklist	uses shared `resume_state` with segment-based save/load helpers

Debugging and Observability

Shim-local debug surfaces

Location	Meaning
`$C83A`	shim log head
`$C980–$C99F`	legacy 32-byte debug ring address range; currently overlaps active shim code
`$C960–$C99F`	`reu_setup_logical`; this replaced the old crash breadcrumb helper slot
`$C980–$C99F`	legacy ring address range; not safe scratch because it overlaps active `reu_setup_logical` bytes

ReadyShell debug surfaces

Location	Meaning
`$C7A0–$C7DF`	ReadyShell RAM debug ring
`$C7F0`	ReadyShell RAM debug head
`0x43F000`	ReadyShell REU debug head
`0x43F010`	ReadyShell REU debug data

The shim-local diagnostics exist for transition-critical breadcrumbs in the always-on resident layer; the ReadyShell diagnostics exist for overlay/runtime analysis inside ReadyShell itself.

Source Priority and Drift Notes

1. `src/boot/boot_asm.s`
2. `src/apps/launcher/launcher.c`
3. `src/lib/resume_state.{h,c}`
4. `src/lib/reu_mgr*.{h,c}`
5. `src/lib/clipboard*.{c,h}`
6. `build_support/memory_map_spec.json`

- Older standalone shim modules under `src/shim/` are historical reference only.
- `src/shim/reu.h` still carries some legacy symbolic bank naming; docs must not mistake that for active clipboard payload behavior or current skipped physical-bank placement.
- Some older high-level banner comments near the top of `boot_asm.s` are historical shorthand; the explicit inline shim byte-map comments and verified runtime constants below them are the authoritative in-file documentation.

Updated for the ReadyOS 0.2.4 documentation sync. The original snapshot values are preserved for historical comparison; current map-backed values use the public 0.2.4 version label. Older shim report variants in this directory are historical snapshots only.